

[Colab notebook](#)

Introduction

In the following case study a series of experimentation was performed on the given dataset ADE20k, in order to get an adequate model that classifies the desired results. The series of experimentation and their results will be discussed in this project that demonstrates ability to choose an appropriate method for investigation a problem and identify the solutions, advantages and limitation of those methods.

Literature Review

The initial tasks are to build an image segmentation model that classifies 4 specific objects i.e. Book, Car, Airplane and Person. The dataset for this task is ADE20K that contains large amount of real-world indoor and outdoor pictures.

Vanilla U-Net

The model used for segmentation is U-net ([Ronneberger et al., 2015](#)). The primary motivation behind choosing this model is due to its design. It is the most suitable to be a baseline model, as its designed for image segmentation. It consists of two main parts, an encoder and a decoder. The encoder is mostly responsible for extracting the hierarchical features from the data, while reducing the dimension of input image, on the other hand a decoder restores the original image resolution and gives out a pixel based segmentation. The direct link between the encoder and decoder, instead of passing information through every layer, it skips some layer and sends important information that's high in value.

DeepLabv3 + Resnet50

The next obvious approach would be to use a U-net model with resnet30 or 50, since we have a data scarcity on a specific class (airplane). One way to fix that would be to train the model using the weights of a pre-trained model which has already seen thousands of airplane image. However there are limited number research done on semantic segmentation with highly imbalanced datasets. ([Ahmed et al., 2025](#)) One such study investigates the similar issue using advanced techniques to improve model performance. Despite being almost similar, the severity of the problem is higher in this project then the one in the paper. The problem discussed in paper is pixel based scarcity of data that is solved by Focal loss.

This research on [DeepLabV3](#) (Liang chen et al., 2017), that tackles 2 main issue with semantic segmentation. First, loss of spatial resolution from repeated pooling in CNN, mentioned above, that what an encoder does, and second, object segmentation of images that appear in different scale. This impressive approach combined with ResNet50 has a great possibility to fix the initial data imbalance issue. There is less pixel loss, with high quality training along with weights that has seen huge numbers of airplane (an other) images.

Data and EDA

The ADE20K dataset has over 100 classes but we are only using 4 classes i.e. Plane, Person, Car and Book for now. Since we have the annotation for the dataset we can easily extract relevant images along with their masks using COCO. There are total 350 images for us to process. The image below is a clean representation of classes and their distributions.

Class	Images
person	233
car	148
book	51
airplane	5

Class	Object Instances
person	1084
car	689
book	155
airplane	13

Class	Total Pixels
person	3,460,166
car	2,111,519
book	266,028
airplane	113,628

Image1: Class distribution

There is clear class imbalance between the data [Image1]. The instances of these the weakest class is also low, along with the pixel count. From this data alone it can be concluded that any normal classifier built from scratch will struggle to detect airplane.

Histogram

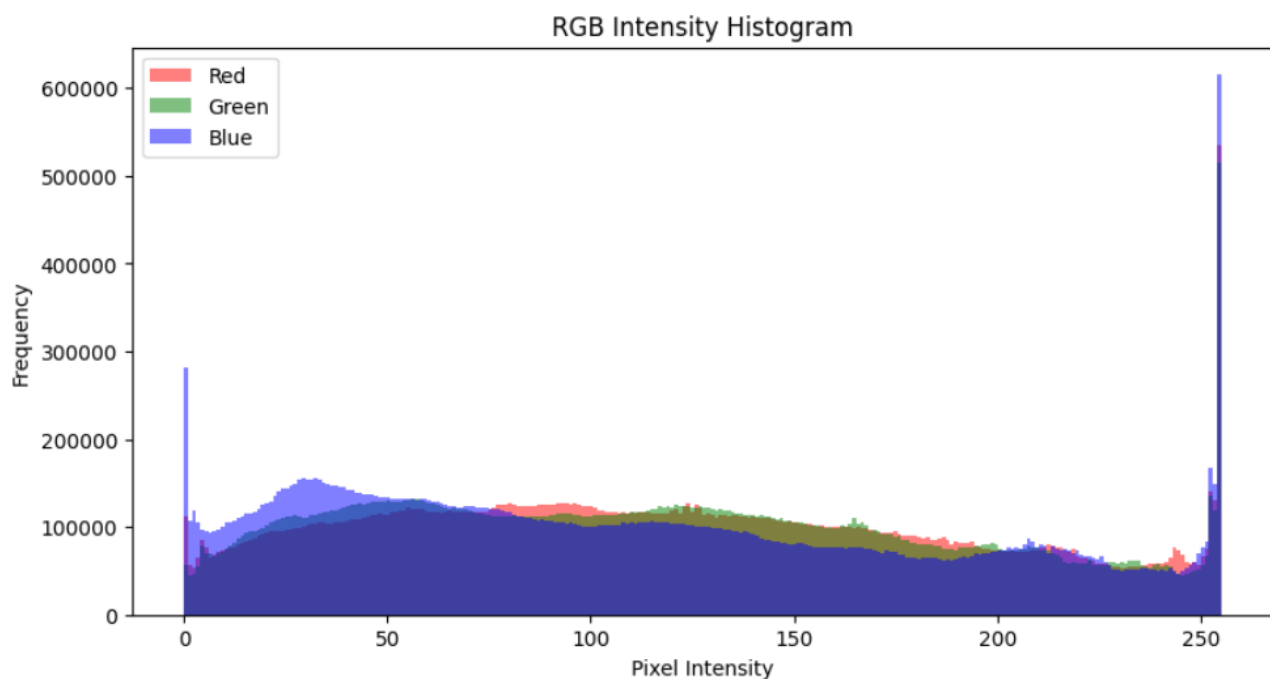


Image2: Colour Channel Intensity

Key takeaways:

The strong spike at 0 and 255 is due to data padding that we did to resize the image properly. The blue channel is dominating at lower intensities, which means the colour is biased slightly towards darker region. Red, green and blue are mildly overlapping each other in mid range, suggest neutral and balanced colour distribution reflecting the natural photographic content.

Heatmap

The dataset has low inter-feature correlation, which means the selected classes are mostly independent. Since there is not strong correlation, the model will learn each class individually rather than relying on co-occurring pattern.

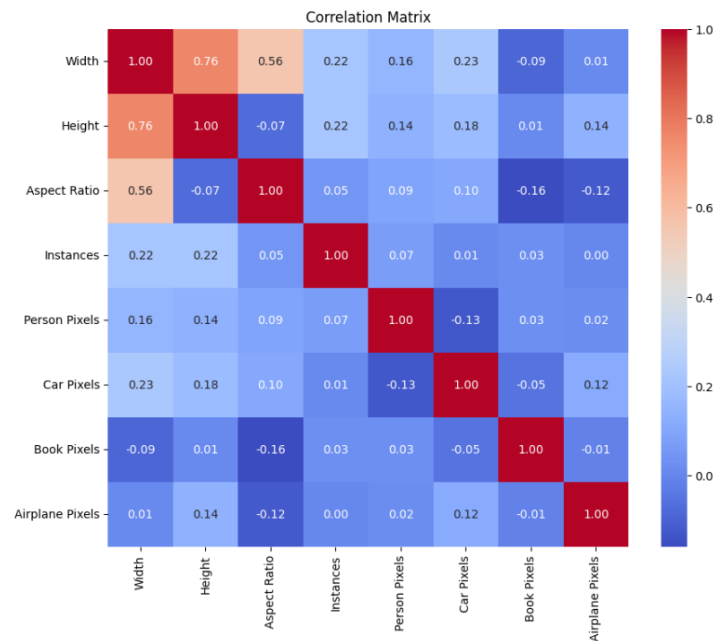


Image 3: Heatmap

Chord Diagram

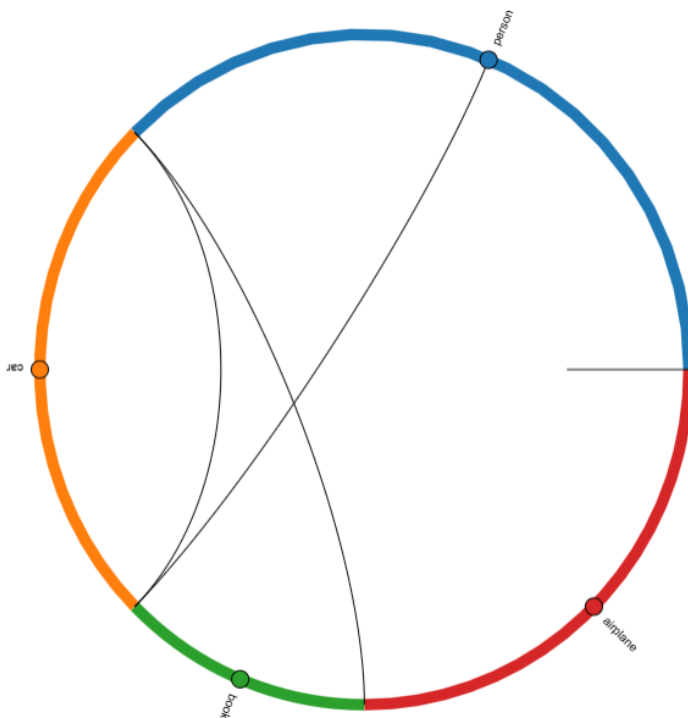


Image 4: Chord Diagram

The image show us that dataset contain uneven class relationship, as person and car are very frequent and book and car are low in numbers. This imbalance will mainly contribute to increase difficulty of learning robustness and generalization

Methodology

Model 1 Trial1:

The baseline model, vanilla U-NET, with double convolution block, simple encoder and decoder, with “Adam” optimizer and “Crossentropy” as loss function was used. This ran on 10 epochs.

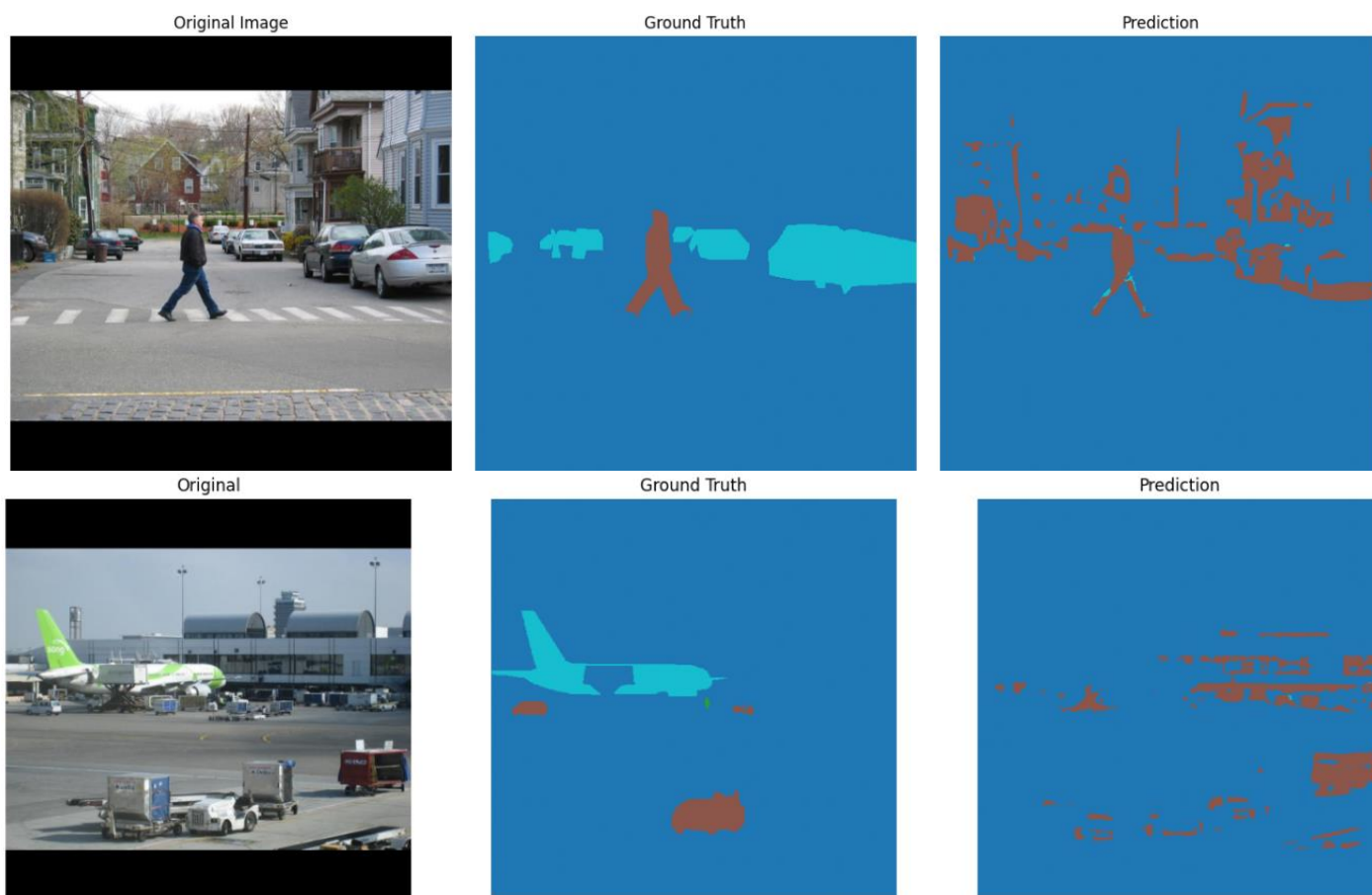


Image 5: Original vs prediction comparison Model1 Trial1

The results are expected, with a dataset with barely any airplane images the model is struggling to detect airplane. Here are evaluation metrics present below.

This tell use a lot of key details where the model is struggling. Pixel accuracy is high but its misleading as the background takes over the accuracy metrics.

```
metrics_1 = evaluate_model(model, val_loader, device)
print_metrics(metrics_1)
```

```
Pixel Accuracy : 0.7637
Mean IoU      : 0.1968
Mean Dice     : 0.2527
```

Background	IoU: 0.7628	Dice: 0.8654
Person	IoU: 0.1010	Dice: 0.1835
Car	IoU: 0.1202	Dice: 0.2146
Book	IoU: 0.0000	Dice: 0.0000
Airplane	IoU: 0.0000	Dice: 0.0000

Based on IoU results we can see the model is barely detecting Person and car, and completely fails to detect Book or airplane, since their values are zero.

This also shows that in semantic segmentation, IOU and dice score is more reliable than accuracy, as it not being dominated by one factor. This can further be improved with more epochs that the second trial.

Image 6: Evaluation metrics Model1, Trial1

Model 2, Trial 2

Simple upgrade, instead of training on 10 epochs now we train on 30, same architecture, loss and optimizer.

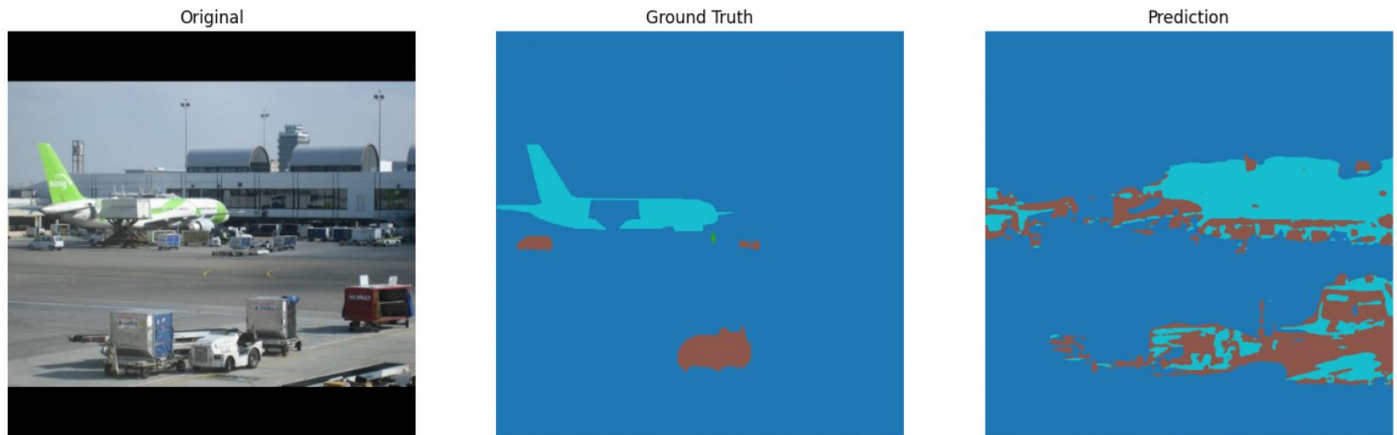


Image 7: Original vs prediction model1, trial 2

The improvement is definitely there but more epochs on top of this is not going to solve the problem.

Pixel Accuracy	:	0.7957
Mean IoU	:	0.1955
Mean Dice	:	0.2429

Background	IoU: 0.8000	Dice: 0.8889
Person	IoU: 0.0919	Dice: 0.1683
Car	IoU: 0.0854	Dice: 0.1574
Book	IoU: 0.0000	Dice: 0.0000
Airplane	IoU: 0.0000	Dice: 0.0000

And despite the pixel accuracy being slightly high, the model completely failed to recognize books and plane class.

A better approach would be use a Dice loss accuracy on top on crossentropy, that is the standarad option for a semantics segmentation option.

Image 8: Evaluation metrics model 1, trial 2

Model 1, Trial 3

Using Dixeloss + crossentropy as loss function, same optimizer, same architecture, trained on 30 epochs.

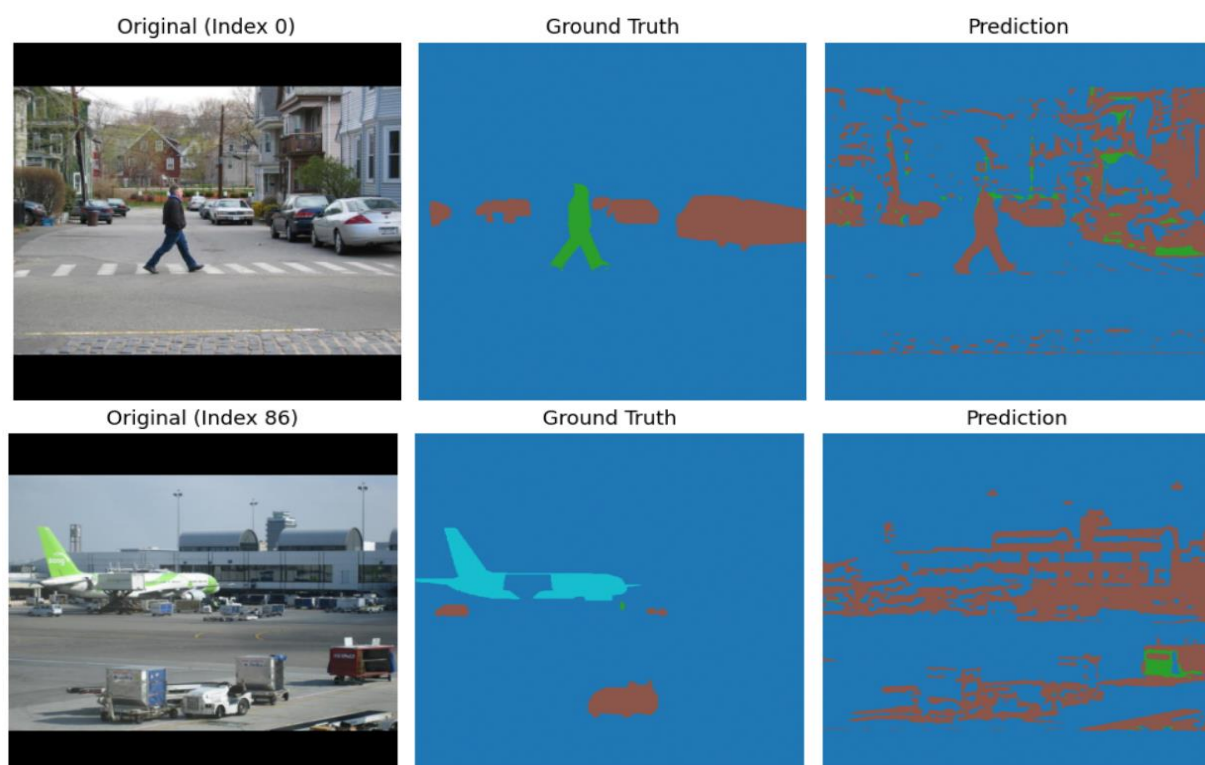


Image 9: Model 1 trial 3, Actual vs prediction comparison

There is a clear improvement with the way the model is learning but the key point is still missing that can be seen in the metrics below.

Pixel Accuracy : 0.7796		
Mean IoU : 0.1772		
Mean Dice : 0.2132		

Background	IoU: 0.7903	Dice: 0.8828
Person	IoU: 0.0481	Dice: 0.0918
Car	IoU: 0.0478	Dice: 0.0913
Book	IoU: 0.0000	Dice: 0.0000
Airplane	IoU: 0.0000	Dice: 0.0000

The metrics show that IOU and Dice score increased compared to previous model, that tell us that model is performing worse than usual on the 2 initial class and still unable to catch the unbalance classes i.e. book and airplane.

The last improvement tactic here would be to implement data augmentation. The problem from the start has been the scarcity of data so this will be the last attempt to increase the volume of data so the

Image 10: Evaluation Metrics
model can see more examples of airplane while training.

Model 1 trial 4

Data augmented and crossentropy and dice loss loss function, with adam optimizer trained on 30 epochs.



Image 11: Original vs Pretrained

The results are unexpected, with data augmentation the expectation were high but, the poor performance can easily be depicted from the image, the model hasn't improved and it shows not signs of capturing book and plane even after data augmentation. Working further on this model is not recommended.

Model 2 trial 1

The best model theoretically so far, DeepLabV3 with a pretrained ResNet50 backbone, which is also a direct comparison to vanilla U-net. The original DeepLabv3 was designed to predict 21 classes, so here the final layer was replaced with a 1v1 cnn layer that gives out 5 classes as output. The rest of the pretrained network along with resnet backbone and ASPP module was trained manually.

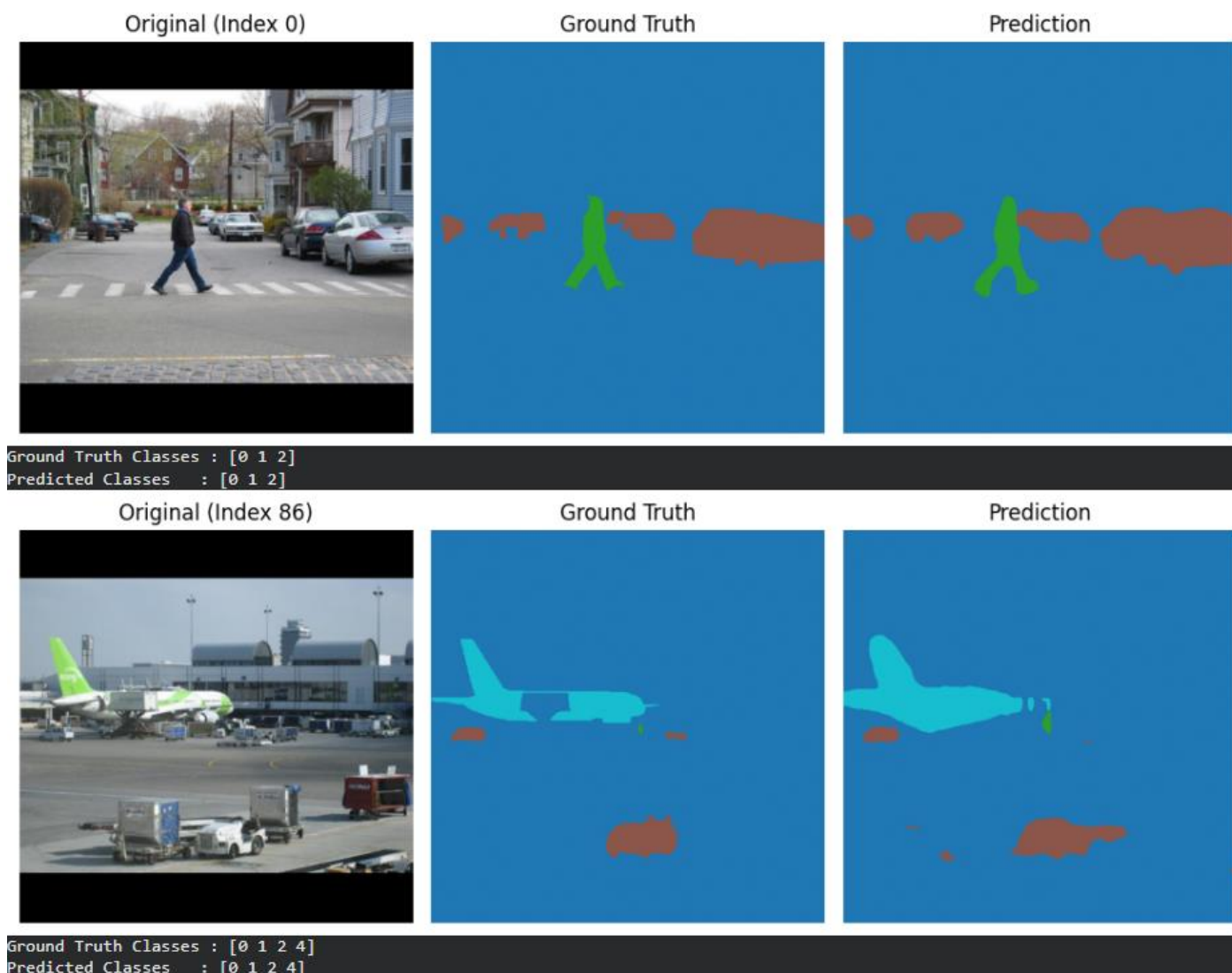


Image 12: Actual vs prediction model 2 trial 1

Pixel Accuracy : 0.9779
Mean IoU : 0.5631
Mean Dice : 0.6742

Background	IoU: 0.9770	Dice: 0.9884
Person	IoU: 0.6856	Dice: 0.8135
Car	IoU: 0.6853	Dice: 0.8133
Book	IoU: 0.2036	Dice: 0.3384
Airplane	IoU: 0.2640	Dice: 0.4177

From the results we can see the model outperforms the initial model and is able to capture the most problematic class. Lets look at the results in details.

The model clearly segments the the airplane with Per class IoU of 26.40 % and a very good dice score of 41.77% showcasing the significant improvement for the most underrepresented class.

Additionally, the predicted mask has better object boundaries and fewer false positives with more accurate localization of all given classes.

This also concludes practically that transfer leaning is an optimal solution when there is scarcity or high class imbalance of data.

References

- 1) [Convolution network Biomedical Image segmentation](#) (Ronneberger et al., 2015)
- 2) [Addressing class imbalance in skin lesion segmentation.](#)(Ahmed et al., 2025)
- 3) [Rethinking Atrous Convolution for Semantic Image Segmentation](#)(Liang Chen et al., 2017)

Appendix

```
# -*- coding: utf-8 -*-
```

```
"""Image_segmentaion.ipynb
```

Automatically generated by Colab.

Original file is located at

<https://colab.research.google.com/drive/1o5LAee5f12EZk3VpN1Zt9HVzCFY2Ucry>

```
# Semanticc Segmentaion Assignment by Sandeep Kumar
```

```
"""
```

```
# Commented out IPython magic to ensure Python compatibility.
```

```
# Importing relevant modules.
```

```
import random
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from pycocotools.coco import COCO # API for COCO dataset.
```

```
import skimage.io as io
```

```
import seaborn as sns
```

```
import torch
```

```
import cv2
```

```
from torch.utils.data import Dataset
```

```
from torch.utils.data import DataLoader
```

```
# %matplotlib inline
```

```
#imports for model 2
```

```
from torchvision.models.segmentation import deeplabv3_resnet50
```

```
from torchvision.models.segmentation import DeepLabV3_ResNet50_Weights
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
print(device)
```

```
# Import and mount Google Drive.
```

```
from google.colab import drive
```

```
drive.mount("/content/drive")
```

```
# Unzip the dataset file and store it in a folder called images.
```

```
!unzip "/content/drive/MyDrive/RMDS_segmentation_dataset_ADE20K_350.zip" -d
```

```
"/content/drive/MyDrive/ADE20K/"
```



```

# Define location of training data (images and labels).

train_data_path = "/content/drive/MyDrive/ADE20K/train"
train_annotation_file = "/content/drive/MyDrive/ADE20K/instances_train.json"

val_data_path = "/content/drive/MyDrive/ADE20K/val"
val_annotation_file = "/content/drive/MyDrive/ADE20K/instances_val.json"

coco = COCO(train_annotation_file)
val_coco = COCO(val_annotation_file)

# Extract and display COCO categories and supercategories for the training dataset.

category_ids = coco.getCatIds()
categories = coco.loadCats(category_ids)

print(categories)

names_cats = [cats["name"] for cats in categories]
print(len(names_cats), "COCO categories:", " ".join(names_cats))

# Define function to extract category name from ID.

def get_category_name(class_ID, categories):
    for i in range(len(categories)):
        if categories[i]["id"] == class_ID:
            return categories[i]["name"]
    return "None"

# Look at one example.

category_name_10 = get_category_name(10, categories)
print(f"The category name is {category_name_10}.")

# Make sure to check out some others.

# Get all training images containing a given object category or categories.

filter_class = ["airplane", "car", "book", "person"]
category_ids = coco.getCatIds(catNms=filter_class)

# Collect image IDs for each category and then combine them to get unique IDs
all_image_ids = set()
for cat_id in category_ids:
    img_ids_for_cat = coco.getImgIds(catIds=[cat_id])
    all_image_ids.update(img_ids_for_cat)

image_ids = list(all_image_ids)

print(f"Number of images containing specified category(ies): {len(image_ids)}.")
print(f"IDs of images containing specified category(ies): {image_ids}.")

# What happens when you are filtering for a different category or multiple classes?

# Load and display example training image with segmentation masks.

```

```

# Re-defining target_classes and related variables as the previous cells were deleted.
target_classes = ["person", "car", "book", "airplane"]
target_classes_IDS = coco.getCatIds(catNms=target_classes)

training_images = []
for class_name in target_classes:
    training_images_categories = coco.getCatIds(catNms=class_name)
    training_images_IDS = coco.getImgIds(catIds=training_images_categories)
    training_images += coco.loadImgs(training_images_IDS)

unique_training_images = []
for i in range(len(training_images)):
    if training_images[i] not in unique_training_images:
        unique_training_images.append(training_images[i])

import random
random.seed(0)
random.shuffle(unique_training_images)

training_image = unique_training_images[50]
print(training_image)

image = io.imread(f'{train_data_path}/{training_image["file_name"]}')
plt.axis("off")
plt.imshow(image)

training_image_annotations_ID = coco.getAnnIds(
    imgIds=training_image["id"], catIds=target_classes_IDS, iscrowd=None
)
training_image_annotations = coco.loadAnns(training_image_annotations_ID)
coco.showAnns(training_image_annotations, draw_bbox=False)

plt.show()

# To generate segmentation mask use annToMask function and extract the info stored in the
# annotations.
# For example, for the first object in our example training image:

mask_example = coco.annToMask(training_image_annotations[0])

print(type(mask_example))
print(mask_example)
print(mask_example.shape)
print(np.max(mask_example))
print(np.min(mask_example))

mask = np.zeros(
    (training_image["height"], training_image["width"]),
    dtype=np.uint8
)

for ann in training_image_annotations:
    object_category = get_category_name(
        ann["category_id"],

```

```

        categories
    )

    pixel_value = target_classes.index(object_category) + 1

    object_mask = coco.annToMask(ann)

    mask[object_mask == 1] = pixel_value

print(f"Unique pixel values in the mask: {np.unique(mask)}")

plt.imshow(mask)
plt.show()

"""# Process towards Baseline model (My code from here)"""

# Helper function to check class distribution
# To find function ctrl + F, then # [serial number of function used]
# Functions that are not reused more than once does not have a serial number

def get_class_distribution(coco, target_classes):
    """
    Prints the number of unique images containing each target class.

    Args:
        coco: COCO object
        target_classes (list): List of class names
    """

    print("-" * 45)
    print(f"{'Class':<15}{'Images':>10}")
    print("-" * 45)

    class_distribution = {}

    for class_name in target_classes:

        # Get category ID
        cat_id = coco.getCatIds(catNms=[class_name])

        # Get image IDs containing this class
        img_ids = coco.getImgIds(catIds=cat_id)

        class_distribution[class_name] = len(img_ids)

        print(f"{'class_name':<15}{'len(img_ids)':>10}")

    print("-" * 45)

    return class_distribution

#2 Heler function to get number of instance
def get_object_distribution(coco, target_classes):
    """
    Count the total number of object instances for each target class.
    """

```

```

print("-" * 50)
print(f"{'Class':<15}{'Object Instances':>20}")
print("-" * 50)

object_distribution = {}

for class_name in target_classes:

    cat_id = coco.getCatIds(catNms=[class_name])[0]

    ann_ids = coco.getAnnIds(catIds=[cat_id])

    object_distribution[class_name] = len(ann_ids)

    print(f"{'class_name':<15}{'len(ann_ids):>20}")

print("-" * 50)

return object_distribution

#3 Helper function to get number of pixels in classes
def get_pixel_distribution(coco, categories, target_classes, image_list):
    """
    Count the total number of pixels belonging to each class.
    """

    pixel_distribution = {cls: 0 for cls in target_classes}

    for image in image_list:

        ann_ids = coco.getAnnIds(imgIds=image["id"])
        anns = coco.loadAnns(ann_ids)

        for ann in anns:

            class_name = get_category_name(
                ann["category_id"],
                categories
            )

            if class_name in target_classes:

                mask = coco.annToMask(ann)

                pixel_distribution[class_name] += mask.sum()

    print("-" * 55)
    print(f"{'Class':<15}{'Total Pixels':>20}")
    print("-" * 55)

    for cls, pixels in pixel_distribution.items():
        print(f"{'cls':<15}{'pixels:>20,}")

    print("-" * 55)

    return pixel_distribution

```

```

target_classes = ["person", "car", "book", "airplane"]

# Class
class_distribution = get_class_distribution(coco, target_classes)

# Object
object_distribution = get_object_distribution(coco, target_classes)

# Pixels
pixel_distribution = get_pixel_distribution(
    coco,
    categories,
    target_classes,
    unique_training_images
)

"""# There is clear class imbalance, airplane lacks lots of data, pixels and instances"""

# Helper function to Resizing the image with black padding
def resize_with_padding(image, target_size=(512, 512), is_mask=False):
    """
    Resize image while maintaining aspect ratio,
    then pad to target size.

    Parameters
    -----
    image : ndarray
    target_size : tuple (width, height)
    is_mask : bool
        True for segmentation masks.

    Returns
    -----
    padded_image
    """

    target_w, target_h = target_size

    h, w = image.shape[:2]

    # Scale factor
    scale = min(target_w / w, target_h / h)

    new_w = int(w * scale)
    new_h = int(h * scale)

    # Choose interpolation
    interpolation = (
        cv2.INTER_NEAREST if is_mask
        else cv2.INTER_LINEAR
    )

    resized = cv2.resize(
        image,
        (new_w, new_h),

```

```

        interpolation=interpolation
    )

    # Create blank canvas
    if len(image.shape) == 3:

        canvas = np.zeros(
            (target_h, target_w, 3),
            dtype=image.dtype
        )

    else:

        canvas = np.zeros(
            (target_h, target_w),
            dtype=image.dtype
        )

    # Center image
    x_offset = (target_w - new_w) // 2
    y_offset = (target_h - new_h) // 2

    canvas[
        y_offset:y_offset + new_h,
        x_offset:x_offset + new_w
    ] = resized

    return canvas

resized_image = resize_with_padding(
    image,
    target_size=(512, 512),
    is_mask=False
)

plt.figure(figsize=(6,6))
plt.imshow(resized_image)
plt.axis("off")
plt.show()

print(resized_image.shape)

#5 Helper function to normalize
def normalize_image(image):
    """
    Normalize image to [0,1].

    Parameters
    -----
    image : ndarray

    Returns
    -----
    image : ndarray
    """

```



```

    image = image.astype(np.float32)
    image /= 255.0

    return image

normalized_image = normalize_image(resized_image)

print(normalized_image.dtype)
print(normalized_image.min())
print(normalized_image.max())

#6 Helper function to convert data to tensor
def to_tensor(image, mask):
    """
    Convert image and mask to PyTorch tensors.
    """

    # HWC -> CHW
    image = np.transpose(image, (2, 0, 1))

    image = torch.tensor(image, dtype=torch.float32)
    mask = torch.tensor(mask, dtype=torch.long)

    return image, mask

image_tensor, mask_tensor = to_tensor(
    normalized_image,
    mask
)

print(image_tensor.shape)
print(mask_tensor.shape)

print(image_tensor.dtype)
print(mask_tensor.dtype)

print(torch.unique(mask_tensor))

"""## Custom dataset that does the preprocessing steps given above on all 350 image"""

#7 Custom dataset class
class ADE20KDataset(Dataset):

    def __init__(
        # Note that this is resuable class and the constructure
        # (__init__) is present so the class is callable
        self,
        # THis resuable code will be helpfull in a pipeline
        coco,
        image_list,
        image_path,
        categories,
        target_classes,
        image_size=(512, 512), # changed several times
        transform=None # changed several times
    ):

        self.coco = coco

```

```

self.image_list = image_list
self.image_path = image_path
self.categories = categories
self.target_classes = target_classes
self.image_size = image_size
self.transform = transform

def __len__(self):

    return len(self.image_list)

def __getitem__(self, index):

    # Image Information
    image_info = self.image_list[index]

    # Load Image
    image = io.imread(
        f"{self.image_path}/{image_info['file_name']}"
    )

    # Load Annotations
    ann_ids = self.coco.getAnnIds(
        imgIds=image_info["id"]
    )

    annotations = self.coco.loadAnns(ann_ids)

    # Create Empty Mask
    mask = np.zeros(
        (image_info["height"], image_info["width"]),
        dtype=np.uint8
    )

    # Fill Mask
    for ann in annotations:

        class_name = get_category_name(
            ann["category_id"],
            self.categories
        )

        if class_name in self.target_classes:

            pixel_value = (
                self.target_classes.index(class_name)
                + 1
            )

            object_mask = self.coco.annToMask(ann)

            mask[object_mask == 1] = pixel_value

```

```

# Resize
image = resize_with_padding(
    image,
    self.image_size,
    is_mask=False
)

mask = resize_with_padding(
    mask,
    self.image_size,
    is_mask=True
)

# Normalize
image = normalize_image(image)

if self.transform is not None:
    augmented = self.transform(image=image, mask=mask)

    image = augmented["image"]
    mask = augmented["mask"]

# Tensor
image, mask = to_tensor(
    image,
    mask
)

return image, mask

# Run the class
train_dataset = ADE20KDataset(
    coco=coco,
    image_list=unique_training_images,
    image_path=train_data_path,
    categories=categories,
    target_classes=target_classes,
    image_size=(512, 512)
)

# Test to see if works
image, mask = train_dataset[0]

print(image.shape)
print(mask.shape)

print(torch.unique(mask))

# Creating a train folder
train_loader = DataLoader(

```

```

        dataset=train_dataset,
        batch_size=2,          # Hyperparameter
        shuffle=True,
        num_workers=0
    )

# Dataloader test
images, masks = next(iter(train_loader))

print(images.shape)
print(masks.shape)

print(images.dtype)
print(masks.dtype)

"""# EDA"""

# RGB histogram plot to see channels intensity distribution
def plot_rgb_histogram(image_list, image_path, sample_size=None):

    red = []
    green = []
    blue = []

    if sample_size is not None:
        image_list = image_list[:sample_size]

    for img in image_list:

        image = cv2.imread(f"{image_path}/{img['file_name']}")

        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        red.extend(image[:, :, 0].flatten())
        green.extend(image[:, :, 1].flatten())
        blue.extend(image[:, :, 2].flatten())

    plt.figure(figsize=(10,5))

    plt.hist(red,
             bins=256,
             color='red',
             alpha=0.5,
             label='Red')

    plt.hist(green,
             bins=256,
             color='green',
             alpha=0.5,
             label='Green')

    plt.hist(blue,
             bins=256,
             color='blue',
             alpha=0.5,
             label='Blue')

```

```

plt.title("RGB Intensity Histogram")

plt.xlabel("Pixel Intensity")

plt.ylabel("Frequency")

plt.legend()

plt.show()

plot_rgb_histogram(
    unique_training_images,
    train_data_path,
    sample_size=100
)

# Heatmap
def create_image_statistics(coco,
                           image_list,
                           categories,
                           target_classes):

    data = []

    for image in image_list:

        width = image["width"]
        height = image["height"]

        aspect_ratio = width / height

        ann_ids = coco.getAnnIds(imgIds=image["id"])
        anns = coco.loadAnns(ann_ids)

        instance_count = len(anns)

        pixel_counts = {
            cls: 0 for cls in target_classes
        }

        for ann in anns:

            category = get_category_name(
                ann["category_id"],
                categories
            )

            if category in target_classes:

                mask = coco.annToMask(ann)

                pixel_counts[category] += mask.sum()

        data.append({
            "Width": width,

```

```

        "Height": height,
        "Aspect Ratio": aspect_ratio,
        "Instances": instance_count,
        "Person Pixels": pixel_counts["person"],
        "Car Pixels": pixel_counts["car"],
        "Book Pixels": pixel_counts["book"],
        "Airplane Pixels": pixel_counts["airplane"]

    })

    return pd.DataFrame(data)

image_stats = create_image_statistics(
    coco,
    unique_training_images,
    categories,
    target_classes
)

image_stats.head()

plt.figure(figsize=(10,8))

sns.heatmap(
    image_stats.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix")

plt.show()

# Chord diagram
import numpy as np

def build_cooccurrence_matrix(coco,
                              image_list,
                              categories,
                              target_classes):

    n = len(target_classes)

    matrix = np.zeros((n, n), dtype=int)

    for image in image_list:

        ann_ids = coco.getAnnIds(imgIds=image["id"])
        anns = coco.loadAnns(ann_ids)

        classes_present = set()

        for ann in anns:

            category = get_category_name(

```

```

        ann["category_id"],
        categories
    )

    if category in target_classes:
        classes_present.add(category)

classes_present = list(classes_present)

for i in range(len(classes_present)):
    for j in range(i, len(classes_present)):

        a = target_classes.index(classes_present[i])
        b = target_classes.index(classes_present[j])

        matrix[a, b] += 1

        if a != b:
            matrix[b, a] += 1

    return matrix

co_matrix = build_cooccurrence_matrix(
    coco,
    unique_training_images,
    categories,
    target_classes
)

print(co_matrix)

!pip install holoviews bokeh
import holoviews as hv
from holoviews import opts

hv.extension("bokeh")

links = []

for i in range(len(target_classes)):
    for j in range(i + 1, len(target_classes)):

        if co_matrix[i, j] > 0:

            links.append(
                (
                    target_classes[i],
                    target_classes[j],
                    co_matrix[i, j]
                )
            )

nodes = hv.Dataset(
    [(c,) for c in target_classes],
    "Class"
)

```



```
chord = hv.Chord((links, nodes))
```

```
chord.opts(  
    opts.Chord(  
        width=700,  
        height=700,  
        labels="Class",  
        cmap="Category10",  
        edge_cmap="Category10",  
        edge_color="source",  
        node_color="Class"  
    )  
)
```

```
"""# Transfer learning baseline model -- Vanilla U-net
```

```
# Prep for Model 1 Trial 1  
"""
```

```
#8 Double convolution block
```

```
import torch  
import torch.nn as nn
```

```
class DoubleConv(nn.Module):
```

```
    def __init__(self, in_channels, out_channels):
```

```
        super().__init__()
```

```
        self.conv = nn.Sequential(  
            nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),  
            nn.ReLU(inplace=True),  
            nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1),  
            nn.ReLU(inplace=True)  
        )
```

```
    def forward(self, x):
```

```
        return self.conv(x)
```

```
#9 Down block
```

```
class Down(nn.Module):
```

```
    def __init__(self, in_channels, out_channels):
```

```
        super().__init__()
```

```
        self.down = nn.Sequential(  
            nn.MaxPool2d(kernel_size=2),  
            DoubleConv(  
                in_channels,  
                out_channels
```

```

        )

    )

    def forward(self, x):

        return self.down(x)

#10 Up block
class Up(nn.Module):

    def __init__(self, in_channels, out_channels):

        super().__init__()

        self.up = nn.Upsample(
            scale_factor=2,
            mode="bilinear",
            align_corners=True
        )

        self.conv = DoubleConv(
            in_channels,
            out_channels
        )

    def forward(self, x1, x2):

        # x1 = decoder feature
        x1 = self.up(x1)

        # concatenate with encoder feature
        x = torch.cat([x2, x1], dim=1)

        return self.conv(x)

```

```

#11 Feature maps to class predictions
class OutConv(nn.Module):

    def __init__(self, in_channels, out_channels):

        super().__init__()

        self.conv = nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=1
        )

    def forward(self, x):

        return self.conv(x)

```

```

#12 Complete U-NET
class UNet(nn.Module):

```

```

def __init__(self, n_channels=3, n_classes=5):

    super().__init__()

    # Encoder
    self.inc = DoubleConv(n_channels, 64)
    self.down1 = Down(64, 128)
    self.down2 = Down(128, 256)
    self.down3 = Down(256, 512)
    self.down4 = Down(512, 1024)

    # Decoder
    self.up1 = Up(1024 + 512, 512)
    self.up2 = Up(512 + 256, 256)
    self.up3 = Up(256 + 128, 128)
    self.up4 = Up(128 + 64, 64)

    # Output
    self.outc = OutConv(64, n_classes)

def forward(self, x):

    # Encoder
    x1 = self.inc(x)
    x2 = self.down1(x1)
    x3 = self.down2(x2)
    x4 = self.down3(x3)
    x5 = self.down4(x4)

    # Decoder
    x = self.up1(x5, x4)
    x = self.up2(x, x3)
    x = self.up3(x, x2)
    x = self.up4(x, x1)

    # Output
    logits = self.outc(x)

    return logits

# Model INIT U-NET fully trained model
model = UNet(    #12 used
    n_channels=3,
    n_classes=5
)

model = model.to(device)

#13 Helper to compute class weights
def compute_class_weights(train_dataset, num_classes=5):

    pixel_counts = torch.zeros(num_classes)

    for _, mask in train_dataset:

```

```

        unique, counts = torch.unique(mask, return_counts=True)

        for cls, count in zip(unique, counts):

            pixel_counts[cls] += count

    print("Pixel Counts")
    print(pixel_counts)

    total_pixels = pixel_counts.sum()

    class_weights = total_pixels / (num_classes * pixel_counts)

    class_weights = class_weights / class_weights.sum() * num_classes

    return class_weights

class_weights = compute_class_weights(
    train_dataset,
    num_classes=5
)

print(class_weights)

# Hyper parameter to pick suitable loss function
criterion = nn.CrossEntropyLoss(
    weight=class_weights.to(device)
)
criterion = criterion.to(device)

# Default for all trials : Adam
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-4
)

# For Early stopping if no improvement
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=3
)

#14 Training helper function, only trian 1 epoch to see output and smooth run
def train_one_epoch(
    model,
    dataloader,
    criterion,
    optimizer,
    device
):

    model.train()

    running_loss = 0.0

```

```

for images, masks in dataloader:

    images = images.to(device)
    masks = masks.to(device)

    # Forward Pass
    outputs = model(images)

    if isinstance(outputs, dict):
        outputs = outputs["out"]

    # Compute Loss
    loss = criterion(
        outputs,
        masks
    )

    # Backpropagation
    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

    # Running Loss
    running_loss += loss.item()

epoch_loss = running_loss / len(dataloader)

return epoch_loss

train_loss = train_one_epoch(
    model,
    train_loader,
    criterion,
    optimizer,
    device
)

print(f"Training Loss : {train_loss:.4f}")

# Get validation images containing the target classes
validation_images = []
for class_name in target_classes:
    validation_images_categories = val_coco.getCatIds(catNms=class_name)
    validation_images_IDs = val_coco.getImgIds(catIds=validation_images_categories)
    validation_images += val_coco.loadImgs(validation_images_IDs)

unique_validation_images = []
for i in range(len(validation_images)):
    if validation_images[i] not in unique_validation_images:
        unique_validation_images.append(validation_images[i])

val_dataset = ADE20KDataset(

```

```

    coco=val_coco,
    image_list=unique_validation_images,
    image_path=val_data_path,
    categories=categories,
    target_classes=target_classes,
    image_size=(512,512)
)

```

```

val_loader = DataLoader(
    val_dataset,
    batch_size=2,
    shuffle=False,
    num_workers=0
)

```

#15 Validation helper function, only vvalidate 1 epoch to see output and smooth run

```

def validate_one_epoch(
    model,
    dataloader,
    criterion,
    device
):
    model.eval()

    running_loss = 0.0

    with torch.no_grad():
        for images, masks in dataloader:

            images = images.to(device)
            masks = masks.to(device)

            outputs = model(images)

            if isinstance(outputs, dict):
                outputs = outputs["out"]

            loss = criterion(
                outputs,
                masks
            )

            running_loss += loss.item()

    epoch_loss = running_loss / len(dataloader)

    return epoch_loss

val_loss = validate_one_epoch(
    model=model,
    dataloader=val_loader,
    criterion=criterion,
    device=device
)

```

```

print(f"Validation Loss : {val_loss:.4f}")

def fit(
    model,
    train_loader,
    val_loader,
    criterion,
    optimizer,
    scheduler,
    device,
    epochs
):
    history = {
        "train_loss": [],
        "val_loss": []
    }

    for epoch in range(epochs):

        train_loss = train_one_epoch(
            model,
            train_loader,
            criterion,
            optimizer,
            device
        )

        val_loss = validate_one_epoch(
            model,
            val_loader,
            criterion,
            device
        )

        scheduler.step(val_loss)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)

        print(
            f"Epoch [{epoch+1}/{epochs}] "
            f"| Train Loss: {train_loss:.4f} "
            f"| Val Loss: {val_loss:.4f}"
        )

    return history

# Model training
history_10 = fit(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,

```

```

        optimizer=optimizer,
        scheduler=scheduler,
        device=device,
        epochs=10
    )

# plot the results
plt.figure(figsize=(8,5))

plt.plot(
    history_10["train_loss"],
    label="Train Loss"
)

plt.plot(
    history_10["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")

plt.legend()

plt.grid(True)

plt.show()

# Get one validation sample
image, mask = val_dataset[0]

# Convert image back to HWC format
image = image.permute(1, 2, 0).numpy()

# Convert mask to numpy
mask = mask.numpy()

plt.figure(figsize=(10,5))

# Original Image
plt.subplot(1,2,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

# Ground Truth
plt.subplot(1,2,2)
plt.imshow(mask, cmap="tab10")
plt.title("Ground Truth Mask")
plt.axis("off")

plt.tight_layout()
plt.show()

# Put model in evaluation mode

```



```

model.eval()

# Get one validation sample
image, mask = val_dataset[0]

# Add batch dimension
input_image = image.unsqueeze(0).to(device)

# Predict
with torch.no_grad():
    output = model(input_image)

prediction = torch.argmax(output, dim=1).squeeze(0).cpu().numpy()

# Convert image back for plotting
image = image.permute(1, 2, 0).numpy()

# Convert mask to numpy
mask = mask.numpy()

# Plot
plt.figure(figsize=(15,5))

# Original
plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original Image")
plt.axis("off")

# Ground Truth
plt.subplot(1,3,2)
plt.imshow(mask, cmap="tab10")
plt.title("Ground Truth")
plt.axis("off")

# Prediction
plt.subplot(1,3,3)
plt.imshow(prediction, cmap="tab10")
plt.title("Prediction")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Ground Truth Classes :", np.unique(mask))
print("Predicted Classes    :", np.unique(prediction))

# Find first validation image containing an airplane
for idx in range(len(val_dataset)):

    image, mask = val_dataset[idx]

    if 4 in torch.unique(mask):

        print(f"Airplane found in image index: {idx}")

```

```

        break

# Specifically evaluate airplane
model.eval()

image, mask = val_dataset[86]

input_image = image.unsqueeze(0).to(device)

with torch.no_grad():
    output = model(input_image)

prediction = torch.argmax(output, dim=1).squeeze().cpu().numpy()

image = image.permute(1,2,0).numpy()
mask = mask.numpy()

plt.figure(figsize=(18,6))

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(mask, cmap="tab10")
plt.title("Ground Truth")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(prediction, cmap="tab10")
plt.title("Prediction")
plt.axis("off")

plt.show()

print("Ground Truth:", np.unique(mask))
print("Prediction  :", np.unique(prediction))

#16 Helper to evaluate model
def evaluate_model(
    model,
    dataloader,
    device,
    num_classes=5
):
    model.eval()

    intersection = np.zeros(num_classes)
    union = np.zeros(num_classes)

    dice_intersection = np.zeros(num_classes)
    dice_total = np.zeros(num_classes)

```

```

correct_pixels = 0
total_pixels = 0

with torch.no_grad():

    for images, masks in dataloader:

        images = images.to(device)
        masks = masks.to(device)

        outputs = model(images)

        if isinstance(outputs, dict):
            outputs = outputs["out"]

        predictions = torch.argmax(outputs, dim=1)

        correct_pixels += (predictions == masks).sum().item()
        total_pixels += masks.numel()

        predictions = predictions.cpu().numpy()
        masks = masks.cpu().numpy()

        for cls in range(num_classes):

            pred_cls = predictions == cls
            mask_cls = masks == cls

            inter = np.logical_and(pred_cls, mask_cls).sum()

            uni = np.logical_or(pred_cls, mask_cls).sum()

            total = pred_cls.sum() + mask_cls.sum()

            intersection[cls] += inter
            union[cls] += uni

            dice_intersection[cls] += inter
            dice_total[cls] += total

pixel_accuracy = correct_pixels / total_pixels

class_iou = intersection / (union + 1e-8)

class_dice = (
    2 * dice_intersection
) / (
    dice_total + 1e-8
)

mean_iou = class_iou.mean()

mean_dice = class_dice.mean()

return {

    "Pixel Accuracy": pixel_accuracy,

```

```

        "mIoU": mean_iou,

        "Mean Dice": mean_dice,

        "Per Class IoU": class_iou,

        "Per Class Dice": class_dice

    }

# Also evaluation metrics
metrics_1 = evaluate_model(
    model,
    val_loader,
    device
)

#17 Helper function to print clean metrics
def print_metrics(metrics):

    print(f"Pixel Accuracy : {metrics['Pixel Accuracy']:.4f}")
    print(f"Mean IoU      : {metrics['mIoU']:.4f}")
    print(f"Mean Dice      : {metrics['Mean Dice']:.4f}")

    print("-"*50)

    class_names = [
        "Background",
        "Person",
        "Car",
        "Book",
        "Airplane"
    ]

    for name, iou, dice in zip(
        class_names,
        metrics["Per Class IoU"],
        metrics["Per Class Dice"]
    ):

        print(
            f"{name:<12}"
            f"IoU: {iou:.4f}      "
            f"Dice: {dice:.4f}"
        )

metrics_1 = evaluate_model(model, val_loader, device)
print_metrics(metrics_1)

"""# Model 1 Trial 2
#### 30 epochs
"""

# reinitialize
model = UNet(

```

```

        n_channels=3,
        n_classes=5
    ).to(device)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-4
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=3
)

history_30 = fit(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    epochs=30
)

model.eval()

image, mask = val_dataset[86]

input_image = image.unsqueeze(0).to(device)

with torch.no_grad():
    output = model(input_image)

prediction = torch.argmax(output, dim=1).squeeze().cpu().numpy()

image = image.permute(1,2,0).numpy()
mask = mask.numpy()

plt.figure(figsize=(18,6))

plt.subplot(1,3,1)
plt.imshow(image)
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(mask, cmap="tab10")
plt.title("Ground Truth")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(prediction, cmap="tab10")

```

```

plt.title("Prediction")
plt.axis("off")

plt.show()

print("Ground Truth:", np.unique(mask))
print("Prediction  :", np.unique(prediction))

metrics_2 = evaluate_model(
    model,
    val_loader,
    device
)

metrics_2 = evaluate_model(model, val_loader, device)
print_metrics(metrics_2)

"""# Model 1 Trial 3: weighted CE + Dice Loss"""

# Initilize the dice loss
class DiceLoss(nn.Module):

    def __init__(self, smooth=1e-6):
        super().__init__()
        self.smooth = smooth

    def forward(self, outputs, targets):

        # Convert logits to probabilities
        probs = torch.softmax(outputs, dim=1)

        # One-hot encode masks
        targets = torch.nn.functional.one_hot(
            targets,
            num_classes=outputs.shape[1]
        ).permute(0,3,1,2).float()

        intersection = (probs * targets).sum(dim=(2,3))

        union = probs.sum(dim=(2,3)) + targets.sum(dim=(2,3))

        dice = (2 * intersection + self.smooth) / (union + self.smooth)

        loss = 1 - dice.mean()

        return loss

# Weighted CE + DIce loss
class CombinedLoss(nn.Module):

    def __init__(self, class_weights):

        super().__init__()

        self.ce = nn.CrossEntropyLoss(
            weight=class_weights

```

```

    )

    self.dice = DiceLoss()

    def forward(self, outputs, targets):

        ce_loss = self.ce(outputs, targets)

        dice_loss = self.dice(outputs, targets)

        return ce_loss + dice_loss

# New crieterion for this trial
criterion = CombinedLoss(
    class_weights.to(device)
)

# Reinit
model = UNet(
    n_channels=3,
    n_classes=5
).to(device)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-4
)

# adding a sscheduler for early stopping
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=3
)

history_ce_dice = fit(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    epochs=30
)

# wrapping prediction vissualization in to function
def visualize_prediction(model, dataset, index, device):

    model.eval()

    image, mask = dataset[index]

    input_image = image.unsqueeze(0).to(device)

```

```

with torch.no_grad():
    output = model(input_image)

    if isinstance(output, dict):
        output = output["out"]

prediction = torch.argmax(output, dim=1).squeeze().cpu().numpy()

image_np = image.permute(1, 2, 0).numpy()
mask_np = mask.numpy()

plt.figure(figsize=(10,4))

# Original
plt.subplot(1,3,1)
plt.imshow(image_np)
plt.title(f"Original (Index {index})")
plt.axis("off")

# Ground Truth
plt.subplot(1,3,2)
plt.imshow(mask_np, cmap="tab10", vmin=0, vmax=4)
plt.title("Ground Truth")
plt.axis("off")

# Prediction
plt.subplot(1,3,3)
plt.imshow(prediction, cmap="tab10", vmin=0, vmax=4)
plt.title("Prediction")
plt.axis("off")

plt.tight_layout()
plt.show()

print("Ground Truth Classes :", np.unique(mask_np))
print("Predicted Classes    :", np.unique(prediction))

visualize_prediction(model, val_dataset, 0, device)

visualize_prediction(model, val_dataset, 86, device)

metrics_3 = evaluate_model(
    model,
    val_loader,
    device
)

metrics_3 = evaluate_model(model, val_loader, device)
print_metrics(metrics_3)

"""# M0del 1 Trial 4: Data Augmentation"""

!pip install -q albumentations

import albumentations as A

```



```

train_transform = A.Compose([

    A.HorizontalFlip(p=0.5),

    A.RandomBrightnessContrast(p=0.5),

    A.Rotate(limit=10, p=0.5),

    A.ShiftScaleRotate(
        shift_limit=0.05,
        scale_limit=0.10,
        rotate_limit=0,
        p=0.5
    )
])

# Creating a new training dataset with new augmented data
train_dataset = ADE20KDataset(
    coco=coco,
    image_list=unique_training_images,
    image_path=train_data_path,
    categories=categories,
    target_classes=target_classes,
    image_size=(512,512),
    transform=train_transform
)

val_dataset = ADE20KDataset(
    coco=val_coco,
    image_list=unique_validation_images,
    image_path=val_data_path,
    categories=categories,
    target_classes=target_classes,
    image_size=(512,512),
    transform=None
)

# Fresh model init for trail 4
model = UNet(
    n_channels=3,
    n_classes=5
).to(device)

# keep CE + DL
criterion = CombinedLoss(
    class_weights.to(device)
)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-4
)

scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(

```

```

        optimizer,
        mode="min",
        factor=0.5,
        patience=3
    )

    history_aug = fit(
        model=model,
        train_loader=train_loader,
        val_loader=val_loader,
        criterion=criterion,
        optimizer=optimizer,
        scheduler=scheduler,
        device=device,
        epochs=30
    )

    visualize_prediction(model, val_dataset, 0, device)

    visualize_prediction(model, val_dataset, 86, device)

    metrics_4 = evaluate_model(
        model,
        val_loader,
        device
    )

    metrics_4 = evaluate_model(model, val_loader, device)
    print_metrics(metrics_4)

    """# Model 2 Trial 1: Deeplabv3 with resnet50"""

    model = deeplabv3_resnet50(
        weights=DeepLabV3_ResNet50_Weights.DEFAULT
    )

    print(model.classifier)

    """# there are 256 features and 21 classifiers so only one layer needs changing"""

    model.classifier[4] = nn.Conv2d(
        in_channels=256,
        out_channels=5,
        kernel_size=1
    )
    model = model.to(device)

    print(model.classifier)

    """# Model 2 trial 1"""

    # Initialte the model
    model = deeplabv3_resnet50(
        weights=DeepLabV3_ResNet50_Weights.DEFAULT
    )

```

```

model.classifier[4] = nn.Conv2d(
    in_channels=256,
    out_channels=5,
    kernel_size=1
)

model = model.to(device)

# same loss CE + DL
criterion = CombinedLoss(
    class_weights.to(device)
)

# same optimizer new learning rate
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=5e-5
)

# same scheduler
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min",
    factor=0.5,
    patience=3
)

# Fit
history_deeplab = fit(
    model=model,
    train_loader=train_loader,
    val_loader=val_loader,
    criterion=criterion,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device,
    epochs=30
)

visualize_prediction(model, val_dataset, 0, device)

visualize_prediction(model, val_dataset, 86, device)

metrics_5 = evaluate_model(
    model,
    val_loader,
    device
)

metrics_5 = evaluate_model(model, val_loader, device)
print_metrics(metrics_5)

```